

Project ESCAPE: Structural Discovery via Entropy in BLT Across Code and Natural Language

Priyanka Agarwal
2024101016

Shashwat Mukadam
2024102008

Divyansh Atri
2024113001

Abstract

We investigate whether entropy-triggered boundaries in the Byte Latent Transformer (BLT) align with syntactic constituent starts beyond a density-matched null. Project ESCAPE compares Python, C++, and English prose. We restore the patcher’s 512-byte sliding window, extract entropies for all 16,300 files, freeze the matching tolerance on a calibration split, and score the held-out split with one-to-one matching against 10,000 permutations of each file’s own patch lengths. Held-out start F1 exceeds the null in all three languages (Python 0.152 vs. 0.032; C++ 0.165 vs. 0.036; English 0.495 vs. 0.147). Against the strongest structure-blind baseline (indentation or word starts), however, BLT underperforms it in all three languages. By constituent type, forced-opener constructs such as `if` and `def` align more strongly than open-ended ones, contrary to our prediction. Under injected code noise, alignment degrades only slightly, while noise lowers bytes per patch. Entropy is a surface statistic, and none of these results is evidence of semantic understanding.

1 Introduction

Byte-level models remove a fixed subword vocabulary, but computation still needs to be allocated across long input sequences. BLT groups bytes into variable-length patches using a small next-byte entropy model (Pagnoni et al., 2025). A boundary at the start of a syntactic constituent could indicate that this allocation tracks structure. Rare identifiers, word starts, and formatting can also induce boundaries, so proximity alone cannot distinguish these explanations.

ESCAPE asks whether entropy-triggered boundaries align with constituent starts more than a null with the same patch-length distribution and boundary count (P1). We also ask whether they

align more at starts than at ends (P2), more in code than in prose (P3), and less for constructs whose opening is forced than for open-ended ones (P4). A further question is whether any extra patching near memory-unsafe C++ persists after identifier controls. These are empirical hypotheses: entropy bounds branching ($H \leq \log B$) but is neither a grammatical branching factor nor a measure of program semantics. This report gives held-out results for P1–P4 against both the null and structure-blind baselines, span IoU as a secondary metric, and robustness to code noise (Objective 4).

2 Background and Related Work

BLT dynamically segments byte sequences so that expensive latent computation operates over patches (Pagnoni et al., 2025). Our analysis concerns its entropy patcher, not downstream generation accuracy. ByT5 demonstrates an alternative byte-level modeling approach (Xue et al., 2022); neither architecture by itself establishes correspondence between computational units and syntax.

We use code AST starts and predicted English constituent starts as external structural references. Python samples come from CodeSearchNet (Husain et al., 2019), and C++ uses a subset of The Stack (Kocetkov et al., 2022). English paragraphs come from WikiText-103 (Merity et al., 2017) and are parsed with benepar (Kitaev and Klein, 2018). These reference structures differ in granularity and reliability, motivating language-specific calibration and reporting.

3 Evaluation Protocol

Byte contract and metric. All positions are frozen UTF-8 byte offsets; spans are half-open. Versioned JSONL interfaces carry sample metadata and hashes, patcher outputs, ground-truth

nodes, and region annotations. Validators check joins, hashes, lengths, patch tiling, and entropy coverage. Entropy at byte i is the model’s global Shannon entropy

$$H_i = - \sum_{v \in \mathcal{V}} p_\theta(v \mid x_{<i}) \log p_\theta(v \mid x_{<i}) \quad (1)$$

over its 260 output symbols, in nats. Predictions are interior boundaries labelled `entropy`. Targets are unique starts of eligible, non-wrapper, positive-length nodes, excluding byte zero and file end. Maximum-cardinality one-to-one matching pairs predictions and targets within k bytes. With T matches, P predictions and G targets, precision is T/P , recall T/G and F1 $2T/(P + G)$, pooled across files. Ends are scored the same way.

Frozen tolerance and null. Each language is split at the file level by a seeded SHA256 rank: 20% calibration, 80% test. Only calibration files evaluate $k \in \{0, 1, 2, 4, 8\}$, and we choose the k maximizing the median per-file F1 margin over the null, ties to the smaller k . The hashed record is verified at scoring, and there is no test-time override. For each file, H_0 shuffles the (length, termination) pairs of all patches except the last, which stays in place so that file end never becomes a scored boundary, and rebuilds the endpoints. This preserves file length, the patch-length multiset and the number of entropy boundaries. Each of 10,000 seeded permutations shuffles every test file independently. We report the null mean, the one-sided permutation value, and paired file-bootstrap 95% intervals (2,000 replicates).

Structure-blind baselines. Under the same targets, k and matching, we score boundaries after every newline, at indentation changes, and at every word start. A structural reading requires BLT to beat these, since most code constituents begin after a line break and most prose constituents begin at a word. We report BLT minus baseline F1 with a paired file bootstrap.

4 Data and Checkpoint

The frozen manifest has 16,300 samples: 8,000 Python functions (CodeSearchNet), 8,000 C++ files (The Stack, smol) and 300 English paragraphs (WikiText-103). All bytes were restored and verified against stored hashes. Calibration used 1,600/1,600/60 files and testing 6,400/6,400/240 (Python/C++/English). The selected tolerances were $k = 0, 0$ and 0 .

We use the Hugging Face conversion `itazap/blt-1b-hf` (revision `91aa6b8e`), loading only its 99.5M-parameter patcher. It runs in float32 with threshold 1.3354 nats, no length cap, and the reference 512-token sliding window and 8,192-token chunks, which the port omits. Extraction ran on four RTX 2080 Ti GPUs of the IIIT-H Ada cluster, and all 16,300 files completed with consistent artifacts. Six of eight original fidelity gates passed; two failed (tokenizer EOS and a random-byte threshold). The amended checks, which use the actual BOS-plus-byte input and require over 90% random-byte boundary density, pass. Rerun on the cluster’s software stack, the amended gate set passed. This establishes agreement with an independent reference implementation, not identity with the original weights.

5 Results

P1: alignment beyond the null. Table 1 gives held-out start alignment. The F1 margin over H_0 is +0.120 [0.117, 0.122] in Python, +0.129 [0.125, 0.134] in C++ and +0.348 [0.341, 0.354] in English. Precision/recall are 0.083/0.822, 0.095/0.646 and 0.342/0.894. A sufficient-but-not-necessary mechanism predicts precision above recall. Precision is below recall in all three languages. BLT proposes many more boundaries than there are targets, so most boundaries fall away from constituent starts.

Baselines. The strongest structure-blind proposer is the indentation baseline for Python (F1 0.394), the word-start baseline for C++ (0.339) and the word-start baseline for English (0.769). BLT underperforms it in all three languages (Python -0.242 [-0.247, -0.237]; C++ -0.173 [-0.181, -0.167]; English -0.274 [-0.280, -0.268]). With the frozen $k = 0$, the newline baseline scores near zero: an indented statement starts after its indentation, not at the byte after the newline, and each English sample is a single paragraph. A margin over H_0 that does not survive this comparison indicates line or word segmentation rather than syntactic structure.

P2 and P3. The start margin exceeds the end margin in all three languages (end margins -0.017, -0.035 and -0.079 for Python, C++ and English). These two margins share predictions and are compared descriptively. Pooled by domain, the code

Language	k	Test files	P	R	F1	H_0 F1	$\Delta F1$ [95% CI]	p
Python	0	6,400	0.083	0.822	0.152	0.032	+0.120 [0.117, 0.122]	$\leq 1e-4$
C++	0	6,400	0.095	0.646	0.165	0.036	+0.129 [0.125, 0.134]	$\leq 1e-4$
English	0	240	0.342	0.894	0.495	0.147	+0.348 [0.341, 0.354]	$\leq 1e-4$

Table 1: Held-out entropy-boundary alignment with constituent starts (P1). k was frozen on the calibration split. H_0 F1 is the permutation mean of the fixed-EOF density-matched null (10,000 permutations); intervals are paired file bootstraps; p is the one-sided permutation value.

Lang.	BLT	NL	Indent	Word	BLT—best [95% CI]
Python	0.152	0.000	0.394	0.245	-0.242 [-0.247, -0.237]
C++	0.165	0.041	0.281	0.339	-0.173 [-0.181, -0.167]
English	0.495	0.000	0.008	0.769	-0.274 [-0.280, -0.268]

Table 2: Start F1 of structure-blind baselines under the same targets, k and matching: boundary after every newline (NL), at indentation changes, and at every word start. The last column compares BLT with the strongest baseline for that language.

Lang.	Type	Opener	Starts	R	H_0 R	ΔR
Python	assignment	open	39	0.872	0.156	+0.715
Python	binary_operator	open	7,276	0.683	0.184	+0.498
Python	call	open	33,643	0.766	0.168	+0.598
Python	expression_statement	open	43,709	0.864	0.169	+0.695
Python	for_statement	det.	2,829	0.934	0.162	+0.772
Python	function_definition	det.	485	0.891	0.166	+0.724
Python	if_statement	det.	10,084	0.903	0.158	+0.745
Python	return_statement	open	7,010	0.798	0.249	+0.550
Python	while_statement	det.	263	0.954	0.161	+0.794
C++	assignment_expression	open	8,078	0.452	0.133	+0.319
C++	binary_expression	open	285,064	0.506	0.144	+0.362
C++	call_expression	open	456,491	0.595	0.138	+0.457
C++	declaration	open	249,479	0.773	0.142	+0.632
C++	expression_statement	open	486,954	0.655	0.138	+0.517
C++	for_statement	det.	21,326	0.823	0.136	+0.686
C++	function_definition	det.	101,613	0.768	0.134	+0.635
C++	if_statement	det.	126,003	0.795	0.146	+0.649
C++	return_statement	open	93,611	0.562	0.144	+0.418
C++	while_statement	det.	4,102	0.942	0.150	+0.792

Table 3: Start recall by constituent type (P4). Types are rematched separately, so rows are not additive. A start shared by several nodes is credited to the outermost one, which is why nested types such as Python assignment have few starts. Opener class follows the empirically checked `taxonomy.json`.

margin (+0.129) is smaller than the prose margin (+0.348), so P3 is not supported; in English, BLT also falls short of the word-start baseline (Table 2). Mean bytes per patch are 5.68, 7.86 and 3.77 (Python, C++, English), with mean within-file BPP variances 69.4, 527.8 and 10.3.

P4 and depth. Averaged over types, the recall margin over H_0 for forced-opener constructs is +0.759 in Python and +0.691 in C++, against +0.611 and +0.451 for open-ended ones (Table 3). The forced-opener margin is larger in both code languages, so the data do not support P4 as stated. One reading is that a start boundary precedes the keyword, where the choice of statement is still open; the forcing we predicted applies to the bytes after it, which starts do not measure. Figure 1

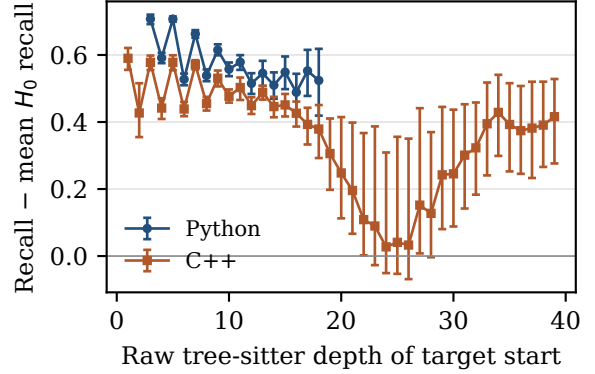


Figure 1: Held-out start recall minus mean H_0 recall by raw tree-sitter depth, for strata with at least 200 target starts. Recall is used because all strata share one prediction set. Raw depth is not comparable across languages. Whiskers are file-bootstrap 95% intervals of recall.

Lang.	Files	BLT	H_0	Line	Word	BLT— H_0 [95% CI]
Python	6,400	0.351	0.342	0.546	0.342	+0.008 [0.007, 0.010]
C++	6,188	0.372	0.356	0.606	0.409	+0.016 [0.013, 0.018]
English	240	0.377	0.334	0.049	0.397	+0.042 [0.040, 0.045]

Table 4: Secondary metric: mean best single-patch IoU over unique constituent spans, held-out split (files with at least one span). H_0 averages 100 fixed-EOF permutations; Line and Word are newline and word-start segmentations.

shows the start recall margin by tree depth.

Secondary: span IoU. For each unique constituent span we take its best IoU with a single patch (Table 4). BLT exceeds the null by +0.008 [0.007, 0.010] in Python, +0.016 [0.013, 0.018] in C++ and +0.042 [0.040, 0.045] in English. In code, line segmentation matches spans far better (0.546 and 0.606). Patches are much shorter than most constituents, so IoU understates alignment by design; it is secondary to start P/R.

Objective 4: robustness to code noise. On 1,000 held-out files per language we inject the four noise types of our proposal, each on its own: missing statement semicolons (C++), a one-space indentation change on 20% of lines, consistent camelCase $\leftrightarrow$ snake_case renaming of 50% of con-

Lang.	Noise	Edits	$\Delta F1$ vs. H_0	Change vs. clean [95% CI]	BPP	Change vs. clean [95% CI]
Python	clean	0	+0.115	–	5.61	–
Python	camel $\leftrightarrow$ snake	4,601	+0.114	-0.001 [-0.002, -0.001]	5.56	-0.05 [-0.06, -0.04]
Python	indentation	4,416	+0.099	-0.016 [-0.018, -0.015]	5.61	+0.00 [-0.00, 0.01]
Python	keyword typo	1,626	+0.112	-0.003 [-0.004, -0.003]	5.38	-0.23 [-0.25, -0.21]
C++	clean	0	+0.131	–	7.88	–
C++	camel $\leftrightarrow$ snake	83,037	+0.129	-0.002 [-0.003, -0.002]	7.78	-0.10 [-0.11, -0.09]
C++	indentation	68,746	+0.118	-0.013 [-0.015, -0.012]	7.75	-0.13 [-0.14, -0.12]
C++	keyword typo	17,884	+0.131	-0.000 [-0.001, 0.000]	7.54	-0.34 [-0.36, -0.31]
C++	missing ;	25,972	+0.129	-0.002 [-0.002, -0.002]	7.82	-0.06 [-0.06, -0.05]

Table 5: Objective 4: robustness to code noise on the same held-out files per language. Each noise type is applied alone; targets are the clean constituents mapped to noised offsets; k is the frozen value. The null uses 1,000 fixed-EOF permutations per file; intervals are paired file bootstraps of noised minus clean.

vertible identifiers, and adjacent-letter swaps in 20% of keywords. Targets are the clean constituents mapped to their noised offsets, so we measure whether boundaries still track the intended structure. The margin over H_0 is +0.115 (Python) and +0.131 (C++) on clean code (Table 5). The largest drop comes from indentation noise in Python (-0.016 [-0.018, -0.015]) and indentation noise in C++ (-0.013 [-0.015, -0.012]); every noised margin stays at or above +0.099 and +0.118. Patching thus degrades gracefully rather than failing. Keyword typos change BPP most (-0.23 and -0.34 bytes per patch), since misspelled keywords are unpredictable and receive more patches.

6 Remaining Work and Conclusion

BLT’s entropy boundaries sit at constituent starts far more often than its own patch lengths would by chance. Relative to the strongest structure-blind baseline, however, BLT underperforms it in all three languages. Any structural claim must therefore be made relative to line and word segmentation, not the null alone. For the final submission we will review the memory-unsafe-region regression (O5) and its region labels, and test the P4 reversal against byte positions after the opening keyword. We will also vary the noise rates for Objective 4. The measurement pipeline, the frozen calibration and the held-out scores reported here are fixed inputs to that work.

Limitations

The checkpoint is a third-party conversion, and the sliding window was restored by us. English targets come from predicted parses, and the English paragraphs were inspected in an earlier exploratory analysis before this protocol was frozen.

About 28% of C++ files trigger parser error recovery; they are included. Python samples are functions that may share repositories, so file bootstraps need not generalize across projects. The null fixes the final patch. Word and newline baselines are deterministic and not density matched. Objective 4 uses one noise rate per type and 1,000 null permutations per file, and span IoU uses 100.

Ethical Considerations

The project analyzes existing text and source code without executing corpus programs. Dataset provenance and licenses accompany restored inputs; CodeSearchNet carries no per-file license. This report draft and supporting implementation used AI assistance and require author review under the course’s disclosure and authorship rules.

References

- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. [CodeSearchNet challenge: Evaluating the state of semantic code search](#). *arXiv preprint arXiv:1909.09436*.
- Nikita Kitaev and Dan Klein. 2018. [Constituency parsing with a self-attentive encoder](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2676–2686, Melbourne, Australia. Association for Computational Linguistics.
- Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. [The stack: 3 TB of permissively licensed source code](#). *arXiv preprint arXiv:2211.15533*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. [Pointer sentinel mixture models](#). In *International Conference on Learning Representations*.

Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinu Iyer. 2025. [Byte latent transformer: Patches scale better than tokens](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258, Vienna, Austria. Association for Computational Linguistics.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. [ByT5: Towards a token-free future with pre-trained byte-to-byte models](#). *Transactions of the Association for Computational Linguistics*, 10:291–306.